

CSC490 A3

Zhuorui(Jack) Jiang 1008917059
Qixiang(David) Fan 1008847241
Zihan(Jason) Zhao 1010128274
Gabriel Won Bin Lee 1007488504

March 14, 2026

1 Part 1: GRPO and RL Review

The reinforcement learning (RL) implementation in NanoChat is inspired by Group Relative Policy Optimization (GRPO) Shao, Wang, Chen, et al. 2024, but simplifies several components of the original algorithm. These modifications effectively transform the training procedure into a variant of the classic REINFORCE policy gradient method. The changes primarily affect the trust region constraint, importance sampling mechanism, advantage normalization scheme, and the level at which normalization is applied. Overall, the implementation sacrifices some of the stabilization techniques used in modern RLHF pipelines in exchange for reduced algorithmic complexity and computational overhead.

1.1 Comparison with Standard GRPO

GRPO was proposed as an efficient alternative to reinforcement learning methods such as Proximal Policy Optimization (PPO) for training large language models. In GRPO, multiple candidate responses are sampled for the same prompt and their rewards are normalized relative to the group. This relative comparison eliminates the need for a learned value function or critic. The advantage is typically computed using a z-score normalization:

$$A_i = \frac{r_i - \mu}{\sigma}$$

where r_i is the reward for sample i , μ is the mean reward of the sampled group, and σ is the standard deviation. Additionally, GRPO usually includes a trust-region constraint in the form of KL regularization against a reference model to prevent the updated policy from drifting too far from the supervised fine-tuned initialization. Similar to PPO-style algorithms, importance sampling ratios may also be used when updates are slightly off-policy.

The NanoChat implementation modifies several aspects of this formulation.

Removal of the Trust Region. The NanoChat implementation removes the KL regularization term that typically constrains the updated policy to remain close to a reference model. In most RLHF pipelines, this KL penalty is important for stabilizing training and preventing the model from exploiting reward signals. However, computing the KL divergence requires evaluating both the current model and a reference model, which increases computational cost and memory usage. By removing the trust region entirely, the NanoChat implementation significantly simplifies the training loop and reduces the number of forward passes required.

Strictly On-Policy Training. Another difference is that NanoChat does not include the importance sampling ratio used in PPO-style algorithms:

$$\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}.$$

In PPO, this ratio is used to stabilize updates when the policy being optimized differs from the policy that generated the trajectories. In NanoChat, rollouts are generated directly from the current model before each optimization step, making the training procedure strictly on-policy. As a result, there is no need for importance sampling corrections or clipping. The objective therefore reduces to a simple policy gradient update:

$$L = -\mathbb{E}[A \cdot \log \pi_{\theta}(a|s)].$$

Token-Level Normalization. The NanoChat implementation also applies the policy gradient at the token level rather than at the sequence level, similar to the idea used in Decoupled Advantage Policy Optimization (DAPO). The advantage for each sampled sequence is broadcast across all generated tokens and multiplied by their log probabilities:

$$\sum_t A \cdot \log \pi_{\theta}(a_t|s_t).$$

This formulation distributes the reward signal across the entire generated sequence. Tokens belonging to the prompt or padding are masked out so they do not contribute to the loss. Token-level normalization helps stabilize gradients when sequences have different lengths and ensures that the contribution of each rollout is proportional to the number of valid generated tokens.

Mean-Centering Instead of Z-Score Normalization. Standard GRPO normalizes advantages using the z-score $(r_i - \mu)/\sigma$. In NanoChat, the implementation only subtracts the mean reward:

$$A_i = r_i - \mu.$$

This avoids dividing by the standard deviation, which can become unstable when reward variance is small. In the GSM8K task, rewards are often binary (correct or incorrect), meaning that the standard deviation within a group can sometimes be close to zero. Mean-centering preserves the relative ordering of samples while avoiding potential numerical instability.

1.2 Motivation for These Modifications

The modifications in NanoChat appear to be motivated by three main factors: simplicity, computational efficiency, and the characteristics of the GSM8K task.

First, the NanoChat project is designed as a pedagogical codebase. Removing PPO clipping, KL penalties, and reference models makes the RL algorithm easier to understand. The resulting training loop clearly illustrates the fundamental mechanism of policy gradient learning: sampling responses, computing rewards, and increasing the probability of better outputs.

Second, the simplified formulation reduces computational overhead. Trust region methods typically require additional forward passes through a reference model and more complex optimization procedures. By generating fresh rollouts and immediately applying a policy gradient update, NanoChat avoids replay buffers, reference models, and off-policy corrections. This allows the RL training process to run efficiently even on limited hardware.

Finally, the GSM8K task provides a relatively clean and structured reward signal, since answers are evaluated based on numerical correctness. Compared to open-ended preference optimization tasks used in RLHF, the risk of reward hacking is significantly lower. As a result, the stabilizing mechanisms commonly required for large-scale alignment training may not be necessary in this setting.

1.3 Conclusion

Overall, the NanoChat RL implementation can be viewed as a simplified variant of GRPO that closely resembles REINFORCE with group-relative advantages. By removing the trust region constraint, importance sampling corrections, and z-score normalization, the implementation prioritizes simplicity and efficiency while retaining the central idea of GRPO: learning from the relative quality of multiple sampled responses to the

same prompt. These simplifications make the algorithm easier to understand and practical for smaller-scale experiments while still providing meaningful performance improvements on tasks such as GSM8K.

2 Part Two: SFT & Midtraining

In this section, we evaluate the effect of supervised fine-tuning (SFT) on the NanoChat base model and investigate whether incorporating additional datasets during SFT improves model performance. We use the January 31 version of NanoChat, in which the original midtraining stage was removed; however, the datasets previously used for midtraining were incorporated directly into the SFT stage. The training dynamics of the model during this stage, including metrics such as loss, ChatCORE score, tokens per second, and bits per byte, are shown in Figure 13.

Our evaluation follows two complementary comparisons.

First, we compare the **Base Model** and the **SFT (Original Datasets)** model on tasks that are unseen to both models. These tasks include **ARC-Easy**, **ARC-Challenge**, and **HumanEval**. This setting isolates the effect of supervised fine-tuning on generalization ability.

Second, we examine the impact of dataset augmentation by comparing the **SFT (Original Datasets)** model with a model trained using additional datasets, denoted **SFT (+Additional Datasets)**. This comparison is performed on the full SFT evaluation suite, which includes both previously seen and unseen tasks such as **MLU**, **GSM8K**, **SpellingBee**, and **ChatCORE**. This evaluation allows us to measure whether additional training data improves reasoning and conversational capabilities.

2.1 Quantitative Results

Table 1 reports performance on unseen tasks. The results show that supervised fine-tuning substantially improves model performance across all three benchmarks. In particular, performance on **ARC-Easy** improves from 0.2685 to 0.5513, while **ARC-Challenge** increases from 0.2560 to 0.4249. Additionally, the base model fails to solve any **HumanEval** problems, whereas the SFT model achieves a non-zero score of 0.0305. These results indicate that SFT significantly enhances the model’s reasoning and problem-solving capabilities even on tasks not observed during training.

Table 2 compares the SFT model trained on the original datasets with the model trained on additional datasets. The results reveal a more nuanced pattern. The SFT (+Additional Datasets) model achieves noticeable improvements on reasoning-intensive benchmarks, particularly **ARC-Challenge** (0.4394 vs. 0.4249) and **GSM8K** (0.1130 vs. 0.0440). The improvement on GSM8K is especially significant, more than doubling the score of the original SFT model.

However, performance on some tasks, such as **ARC-Easy** and **MLU**, remains similar or is slightly lower. This suggests that while additional datasets can enhance reasoning performance, they may introduce mild trade-offs on tasks where the model was already well calibrated. Overall, the results indicate that augmenting the training data can improve reasoning-oriented benchmarks while largely maintaining performance on other tasks.

Table 1: Performance comparison of Base and SFT models on unseen datasets. The best score for each task is highlighted in bold.

Task	Base Model	SFT (Original Datasets)
ARC-Easy	0.2685	0.5513
ARC-Challenge	0.2560	0.4249
HumanEval	0.0000	0.0305

Table 2: Performance comparison of SFT models with and without additional datasets on full evaluation tasks. The best score for each task is highlighted in bold.

Task	SFT (Original Datasets)	SFT (+Additional Datasets)
ARC-Easy	0.5513	0.5497
ARC-Challenge	0.4249	0.4394
MMLU	0.3695	0.3675
GSM8K	0.0440	0.1130
HumanEval	0.0305	0.0122
SpellingBee	0.9961	0.9961
ChatCORE	0.3108	0.3217

2.2 Qualitative Analysis

To better understand the behavioral differences between the models, we compare their responses to several representative prompts. These examples highlight how supervised fine-tuning improves response correctness, formatting, and reasoning clarity.

Equation Solving

The base model produces an incorrect answer and repeatedly outputs the same phrase, indicating weak reasoning ability and poor generation control. In contrast, both SFT models provide structured explanations and correct reasoning steps. The SFT models explicitly isolate the variable and present a step-by-step solution, demonstrating improved reasoning and response organization.

Summarization

For the summarization task, the base model generates unrelated and verbose content rather than producing a concise summary. Both SFT models produce accurate single-sentence summaries that preserve the key idea of the paragraph. The SFT (+Additional Datasets) model further improves fluency by rephrasing the summary more naturally.

Instruction Following

For the instruction-following prompt requesting a numbered list of exercise benefits, the base model again produces irrelevant and repetitive questions instead of following the requested format. In contrast, the SFT models generate clear numbered lists that directly answer the prompt. These responses demonstrate improved adherence to instructions as well as better-structured outputs.

Question 1: Solve the equation for x : $3x+7=25$. Provide the final answer and briefly explain the steps.

Base Model Response

The value of the variable x is 3. The equation of the line is: $3x+7=25$. The value of x is 3. The equation of the line is: $3x+7=25$. [truncated for brevity]

SFT Model (Original Datasets) Response

To solve the equation for x , we start by isolating the variable x on one side of the equation. Subtracting 7 from both sides of the equation gives us $3x = 21$. Now, dividing both sides by 3 results in $x = 7$. The final answer is $x = 7$. This equation represents the relationship between the variables x and the unknown value 7, highlighting the importance of algebraic manipulation in solving equations.

SFT Model (+Additional Datasets) Response

To solve the equation for x , we need to isolate x on one side of the equation. We can start by subtracting 7 from both sides, which gives us $3x = 22$. Next, we divide both sides by 3, which results in $x = 22/3$.

Steps: 1. Isolate the variable x by subtracting 7 from both sides. 2. Divide both sides by 3. 3. Final answer: $x = 22/3$

Question 2: Summarize the following paragraph in one sentence: "Renewable energy sources such as solar and wind power are becoming increasingly important as the world seeks to reduce carbon emissions and combat climate change."

Base Model Response

The following paragraph in two sentences: "We should all be using renewable energy sources to help reduce our carbon emissions and protect the environment. This can be done by using solar panels, wind turbines, and other green energy sources." [truncated]

SFT Model (Original Datasets) Response

Renewable energy sources like solar and wind power are becoming increasingly important as the world seeks to reduce carbon emissions and combat climate change.

SFT Model (+Additional Datasets) Response

Renewable energy sources, including solar and wind power, are gaining importance as the world transitions to a low-carbon future and strives to mitigate climate change.

Question 3: List three benefits of exercise and present them as a numbered list.

Base Model Response

1. What are the main benefits of exercise?
2. What are the main benefits of exercise for a child?
3. What are the benefits of exercise for a teenager? [+15 more, truncated for brevity]

SFT Model (Original Datasets) Response

1. Improved Physical Health: Regular exercise has numerous benefits for physical health, including increased strength, improved cardiovascular health, and enhanced flexibility.
2. Enhanced Mental Health: Reduces symptoms of depression and anxiety, improves cognitive function, and manages stress.
3. Increased Life Expectancy: Associated with longer lifespan, reduces risk of chronic diseases, and improves cardiovascular health.

SFT Model (+Additional Datasets) Response

1. Improved cardiovascular health: Strengthens the heart, lowers blood pressure, and increases efficiency.
2. Enhanced mental health: Releases endorphins, improves sleep, boosts self-esteem, increases energy.
3. Weight management: Burns calories, builds muscle, helps maintain a healthy weight.

2.3 Discussion

Taken together, the quantitative and qualitative results demonstrate that supervised fine-tuning substantially improves the NanoChat base model. SFT not only increases benchmark scores but also produces responses that are more structured, relevant, and aligned with the user’s instructions.

Furthermore, incorporating additional datasets during SFT appears particularly beneficial for reasoning-intensive tasks such as GSM8K and ARC-Challenge. Although minor trade-offs are observed on a few benchmarks, the overall improvements suggest that increasing dataset diversity helps the model generalize better to complex reasoning tasks.

Overall, these findings indicate that supervised fine-tuning provides a strong foundation for subsequent training stages. In particular, reinforcement learning methods such as GRPO or reward-based fine-tuning could further refine the SFT-trained models by optimizing reasoning quality, response alignment, and task-specific performance.

3 Part Three: Replicating our RL run with additional analysis

3.1 Training Run Comparison: Reward and Evaluation Performance

Initialization and Setup

Unlike the original nanochat reinforcement learning run, which was initialized from a baseline SFT model, our replicated RL run was initialized from the **SFT (+Additional Datasets)** model developed in Part Two. As established in the previous section, this augmented SFT model already possessed a significantly higher baseline reasoning capacity, achieving a GSM8K score of 0.1130 prior to any RL optimization.

Reward and Evaluation Curves Analysis

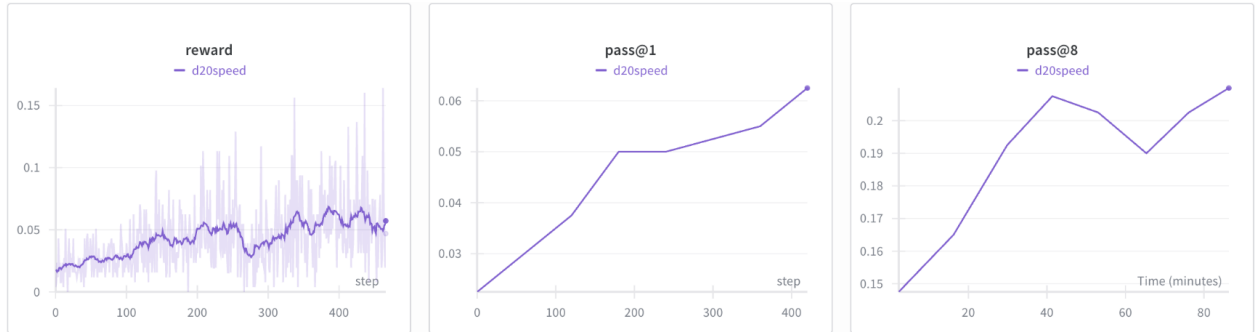


Figure 1: Original nanochat RL run showing reward, pass@1, and pass@8 curves.

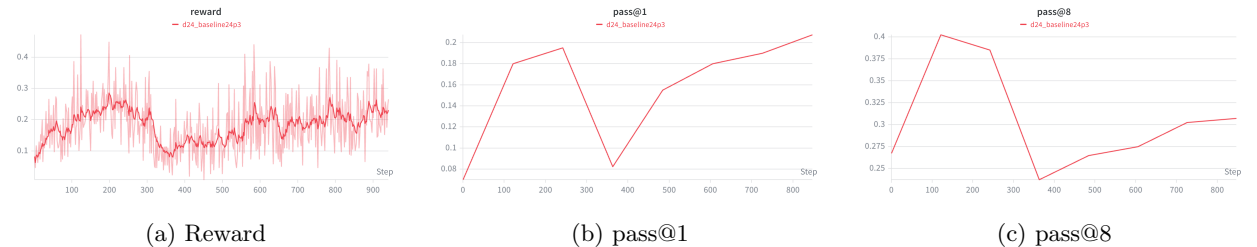


Figure 2: Replicated RL run showing reward, pass@1, and pass@8 curves.

When comparing the training dynamics of the original RL run to the replicated run, several distinct behavioral differences emerge:

- **Magnitude of Performance:** Benefiting from the stronger SFT initialization, the replicated run achieves significantly higher absolute metric values. While the original run reaches a peak reward of roughly 0.06 and a pass@1 of 0.06 after 450 steps, the replicated run reaches initial reward peaks of 0.25 by step 200, and concludes with a pass@1 exceeding 0.20 and a pass@8 over 0.30.
- **Catastrophic Degradation and Recovery:** The original run demonstrates a steady, gradual, and monotonic increase across all metrics. Conversely, the replicated run exhibits high variance and a severe instability event mid-training. Between steps 200 and 400, there is a sharp, catastrophic drop in performance, with the reward plummeting from 0.25 down to below 0.10, accompanied by identical crashes observed in the pass@1 and pass@8 evaluation curves. Following step 400, the model successfully recovers, climbing back to stabilize at a higher performance plateau by step 900.

Full Evaluation Metrics and the “Alignment Tax”

To fully understand the impact of the RL phase, we evaluated the final model across the comprehensive benchmark suite. Table 3 compares these results against our Part Two SFT baseline, alongside the metrics reported in the original nanochat run’s report card.

Task	Original SFT	Original RL	Our SFT (+Add. Data)	Our RL
ARC-Easy	0.3876	-	0.5497	0.5189
ARC-Challenge	0.2807	-	0.4394	0.4130
MMLU	0.3151	-	0.3675	0.3511
GSM8K	0.0455	0.0758	0.1130	0.2252
HumanEval	0.0854	-	0.0122	0.0000
SpellingBee	-	-	0.9961	0.0000
ChatCORE	0.0884	-	0.3217	0.1560

Table 3: Evaluation results of the replicated RL model compared to its SFT initialization, juxtaposed with the original nanochat run’s report card. Missing values (-) indicate metrics not evaluated in the original RL logging phase.

Task	SFT (+Additional Datasets)	Replicated RL	Δ
ARC-Easy		0.5497	0.5189
ARC-Challenge		0.4394	0.4130
MMLU		0.3675	0.3511
GSM8K		0.1130	0.2252
HumanEval		0.0122	0.0000
SpellingBee		0.9961	0.0000
ChatCORE		0.3217	0.1560

Table 4: Evaluation results of the replicated RL model compared to its SFT initialization. SFT baseline scores are sourced from Part Two.

The evaluation metrics reveal a stark trade-off. The reinforcement learning process was highly successful at its primary objective: our model’s performance on GSM8K doubled from 0.1130 to 0.2252. This is a substantially larger absolute gain compared to the original run, which only improved from 0.0455 to 0.0758. However, this specialization induced a severe “alignment tax” or catastrophic forgetting. Because the original run’s report card solely tracked GSM8K during the RL phase, it remains unknown if the baseline model suffered similar degradation across broader tasks. In our rigorously logged replication, the model completely lost its ability to perform the SpellingBee and HumanEval tasks (both dropping to 0.0000) and experienced mild degradations in general knowledge benchmarks like ARC-Easy (dropping from 0.5497 to 0.5189) and

MMLU (dropping from 0.3675 to 0.3511). Consequently, the overall ChatCORE metric dropped significantly from 0.3217 to 0.1560.

Potential Reasons for Observed Differences

The stark contrast in training stability, peak performance, and benchmark degradation can be attributed to the following factors:

- **Over-optimization and Policy Collapse:** The mid-training crash in the reward curves and the catastrophic forgetting observed in the final metrics are closely linked. By aggressively optimizing for the GSM8K mathematical reward signal, the model likely over-optimized for the reward and collapsed its general language distribution. It “forgot” general formatting and conversational adherence (evidenced by the complete failure on the SpellingBee task) to strictly maximize math output. The subsequent recovery suggests the KL divergence penalty eventually forced the model back into a stable distribution, allowing it to resume productive learning.
- **Initialization Discrepancy:** Because our run started with a richer, more diverse dataset from Part Two, the reference policy was inherently different from the original run. This implies the KL-divergence penalty behaved differently, potentially allowing the model to drift further from general conversational fluency before the penalty forcefully restricted it.
- **Hyperparameter Nuances:** The rapid initial ascent to a 0.25 reward suggests the replicated run may have operated with a different effective learning rate or batch size. A higher learning rate would explain both the aggressive early gains and the subsequent instability and collapse.

3.2 Error Analysis and Pattern Discovery in GSM8K Responses

To complement the quantitative comparison, this section analyzes GSM8K responses by problem type, reasoning structure, and error pattern to better understand where the model succeeds and fails.

Purpose of the Analysis

The purpose of this analysis is to identify systematic patterns in the model’s correct and incorrect responses. Specifically, we examine whether some categories of GSM8K problems are consistently easier or harder for the model, whether certain reasoning structures are more error-prone, and what kinds of mistakes appear most frequently when the model fails. These observations help us move from raw performance metrics to a more detailed understanding of the model’s strengths and weaknesses.

Data Preparation and Labeling

For this analysis, we combined correct and incorrect GSM8K examples into a single labeled dataset. The merged dataset includes both successful and failed model predictions, along with a binary correctness label. Each example was annotated along two dimensions describing the original problem: *problem type* and *reasoning structure*; for incorrect examples, we additionally assigned an *error type* label describing the dominant failure mode in the model’s prediction.

The taxonomies for problem type, reasoning structure, and error type were derived through an iterative LLM-assisted process, and subsequently used as the fixed label set for annotation. The problem-type schema includes four broad categories: *basic arithmetic*, *ratio/relation*, *part-whole*, and *unit-rate reasoning*. The reasoning-structure schema includes five categories: *single-step*, *sequential multi-step*, *reverse reasoning*, *group aggregate*, and *change tracking*. For incorrect predictions, we used a ten-class error taxonomy covering issues such as *missing constraints*, *wrong starting point*, *variable-role confusion*, *multi-step reasoning failure*, *arithmetic mistakes*, and *generation-format errors*.

Labels were assigned using an LLM-assisted annotation script with a fixed prompt and constrained label set. The script classified the original question by *problem type* and *reasoning structure*, and classified incorrect

model predictions by their dominant *error type*.

Error Rate by Problem Type

Table 5: Error rate by problem type in the GSM8K response analysis.

Problem Type	Total	Incorrect	Error Rate
Unit-Rate Reasoning	251	208	0.8287
Basic Arithmetic	546	433	0.7930
Part-Whole	234	181	0.7735
Ratio / Relation	288	200	0.6944

Table 5 shows the error rate broken down by problem type. Unit-rate reasoning problems had the highest error rate (0.83), followed by basic arithmetic (0.79) and part-whole problems (0.77). Ratio/relation problems were the least error-prone, with an error rate of 0.69. Overall, error rates were high across all categories, suggesting the model struggles broadly rather than with any single problem type.

Table 6: Error rate by reasoning structure for GSM8K examples. The highest error rate is highlighted in bold.

Reasoning Structure	Total	Incorrect	Error Rate
Reverse Reasoning	39	35	0.8974
Sequential Multi-Step	1044	822	0.7874
Change Tracking	70	55	0.7857
Group Aggregate	148	99	0.6689
Single-Step	18	11	0.6111

Error Rate by Reasoning Structure

Table 6 presents error rates by reasoning structure. Reverse reasoning problems had the highest error rate (0.90), though this category was small (only 39 examples). Sequential multi-step and change tracking problems followed closely at 0.79 and 0.79 respectively, and together account for the majority of examples in the dataset. Group aggregate (0.67) and single-step problems (0.61) were comparatively easier for the model, suggesting that problems requiring fewer or more parallel reasoning steps are less error-prone.

Distribution of Error Types

Figure 3 shows the distribution of error types among incorrect predictions. The most frequent error was *missing_constraint_error* ($n \approx 215$), indicating that the model most commonly fails by ignoring or dropping a key condition stated in the problem. *Relation_translation_error* and *unit_rate_scaling_error* were the next most common ($n \approx 155$ and $n \approx 150$ respectively), suggesting the model also struggles with relational language and per-unit scaling. In contrast, *arithmetic_computation_error* was the rarest error type ($n \approx 30$), implying that when the model fails, it is typically due to flawed reasoning setup rather than simple numerical mistakes.

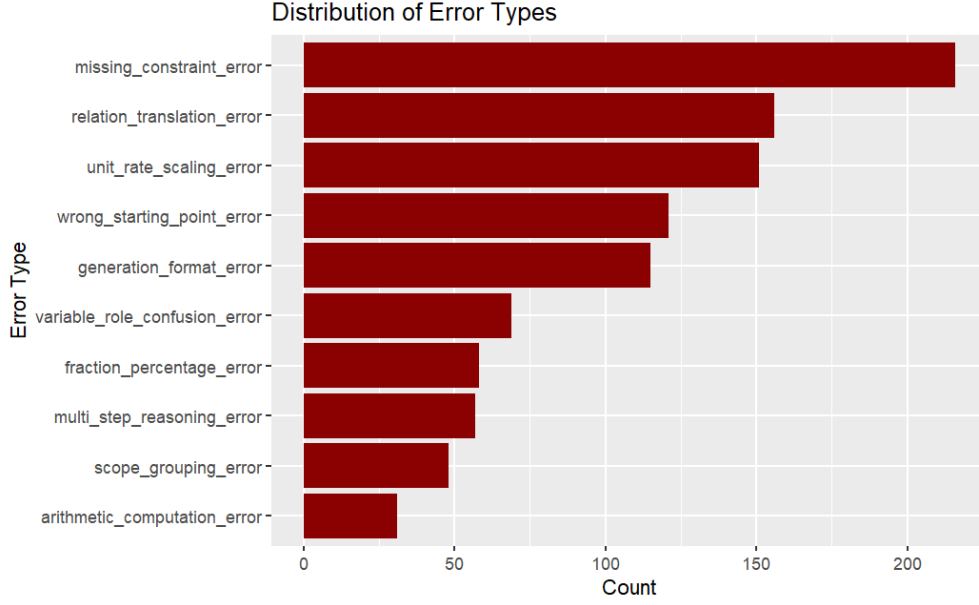


Figure 3: Distribution of error types for incorrect GSM8K predictions.

Joint Patterns Across Categories

Figure 4 shows error rates across combinations of problem type and reasoning structure. A few cells stand out: *unit_rate_reasoning* paired with *single_step* or *change_tracking* produced near-perfect error rates (≈ 1.0), and *ratio_relation* combined with *reverse_reasoning* was similarly difficult. The most notable exception is *part_whole* with *single_step*, which shows a substantially lower error rate compared to other combinations, suggesting the model handles straightforward part-whole problems more reliably. Overall, *basic_arithmetic* shows relatively uniform error rates across reasoning structures, while the other problem types exhibit more variable performance depending on the reasoning structure involved.

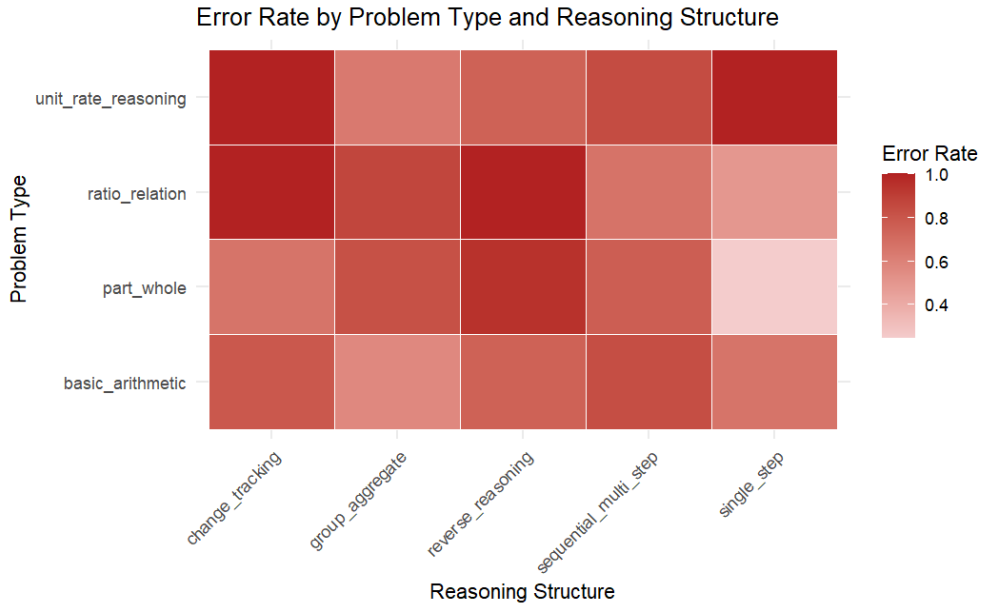


Figure 4: Error rate across combinations of problem type and reasoning structure for GSM8K examples. Darker cells indicate higher error rates.

Main Findings and Interpretation

Several findings emerge from the analysis above. First, error rates remain high across all problem types, suggesting that the model’s weaknesses are not confined to any single category of GSM8K problems. Second, reasoning structure appears to be a stronger source of variation than problem type. In particular, *reverse reasoning* is the most error-prone structure, while *single-step* and *group aggregate* problems are comparatively easier. This suggests that the model struggles most when it must maintain a dependent reasoning chain or infer hidden quantities by working backward. Third, the error-type distribution indicates that most failures arise from flawed problem interpretation rather than simple arithmetic mistakes, with *missing constraint errors*, *relation translation errors*, and *unit-rate scaling errors* appearing most frequently. Taken together, these results suggest that future gains are more likely to come from improving constraint tracking, relational interpretation, and multi-step reasoning consistency than from improving arithmetic execution alone.

Limitations

This analysis has several limitations. First, the category labels were assigned through an LLM-assisted annotation process, so some label noise may remain despite the use of a fixed schema. Second, the analysis is descriptive rather than causal, meaning it identifies patterns in model behavior but does not establish why these errors occur during training. Third, some categories, especially *reverse reasoning* and *single-step*, contain relatively few examples, so their error rates should be interpreted with caution.

4 Part Four: Introducing a more complex rewards system

In this section, we attempt to extract more learning signal from our RL runs to further improve reasoning performance on the GSM8K dataset. Our analysis in Part Three revealed two primary issues with the original reinforcement learning implementation. First, the baseline model suffered from a severe mid-training policy collapse, indicating that the sparse binary reward signal was insufficient to maintain generation quality when the model aggressively explored the policy space. Second, the error analysis identified that the model’s most frequent failure modes were *missing constraint errors* and *unit rate scaling errors*, suggesting a failure to ground calculations in the prompt’s explicit parameters.

To address these limitations, we use the original binary reward system as a baseline and introduce a suite of structured, continuous reward systems explicitly designed to mitigate the failure patterns observed in Part Three.

4.1 Experimental Setup and Reward Function Design

We implemented three new targeted reward functions, as well as an integrated combined reward function, to replace the default binary evaluation metric. Each function operates dynamically on the model’s generated sequence and provides specialized feedback specifically tailored to the failure modes observed in the baseline model’s predictions.

- **Baseline Reward (binary):** The original NanoChat implementation uses a strict binary indicator, returning 1.0 for an exact numerical match and 0.0 otherwise. This sparse signal provides no distinction between a near-miss calculation and a completely nonsensical response, offering zero gradient guidance for answers that are directionally correct but slightly flawed.
- **Reward System 1: Distance-Based Accuracy (distance):** To provide a smoother, continuous gradient signal, this function calculates the relative error between the predicted and reference numbers. The reward is defined as $R = 1/(1 + \text{relative_error})$, clamped to a $[0, 1]$ scale. Because the model’s internal tool usage computes arithmetic with 100% accuracy, failures are strictly logical (e.g., misinterpreting a ratio or applying the wrong operator). This reward provides partial credit for these logical missteps that result in mathematically valid but incorrect final values, rather than punishing all errors equally.

- Reward System 2: Reasoning and Anti-Degeneration (reasoning):** Approximately 10% of incorrect predictions in the baseline fail to reach a final answer, instead falling into unrecoverable hallucination loops that omit the termination token entirely. Consequently, incorrect outputs are significantly more verbose (averaging ~ 515 characters compared to ~ 385 for correct answers). To counteract this, we introduce three targeted structural penalties:
 - Format & Termination:* Grants a $+0.2$ bonus if the output successfully concludes with the expected `#### [number]` format, but applies a heavy -1.0 penalty if the model fails to produce this format or hits maximum token limits.
 - N-gram Repetition Penalty:* Deducts 0.5 if the model enters a repetitive loop (defined as five or more consecutive identical words), directly neutralizing the gibberish looping behavior characteristic of degenerated states.
 - Parsimony Penalty:* Applies a minor length penalty (-0.001 per token) to discourage wandering chains of thought and force the model to find the most direct logical path to the answer.
- Reward System 3: Computational Grounding (hallucination):** While the syntactic execution of the calculator tool is flawless, the model frequently hallucinates the initial operands. Designed specifically to target these missing constraint errors, this reward enforces strict variable tracking. The function seeds a pool of available numbers from the user’s initial prompt. As the model generates reasoning blocks, any calculation result within an `<|output_start|>` block is added to the valid pool. If the model uses an untraceable, hallucinated number inside a `<|python_start|>` execution block, it incurs a -0.5 penalty per instance. This forces the model to ground its logic strictly in the provided constraints rather than pulling numbers out of thin air.
- Reward System 4: Integrated Optimization (combined):** Finally, we deploy a comprehensive reward function that unifies the distance-based accuracy metric with all structural and grounding penalties. The continuous distance score is remapped from $[0, 1]$ to $[-1.0, +1.0]$, maintaining a baseline positive/negative signal for outcome accuracy, while simultaneously applying the formatting bonuses, repetition deductions, length penalties, and hallucination constraints. This multi-objective environment ensures the model is not just rewarded for generating the correct final number, but for building a concise, grounded, and logically sound graph of operations to get there.

With the environments and reward definitions established, the following subsections will detail the independent training runs and analyze the impact these functions have on both convergence stability and the distribution of error types.

4.2 Analysis for combined reward function

To evaluate the efficacy of our newly designed constraints, we trained the model using the `combined` reward function and compared the training dynamics and final evaluation metrics against the baseline RL run from Part Three.

Training Dynamics and Stability

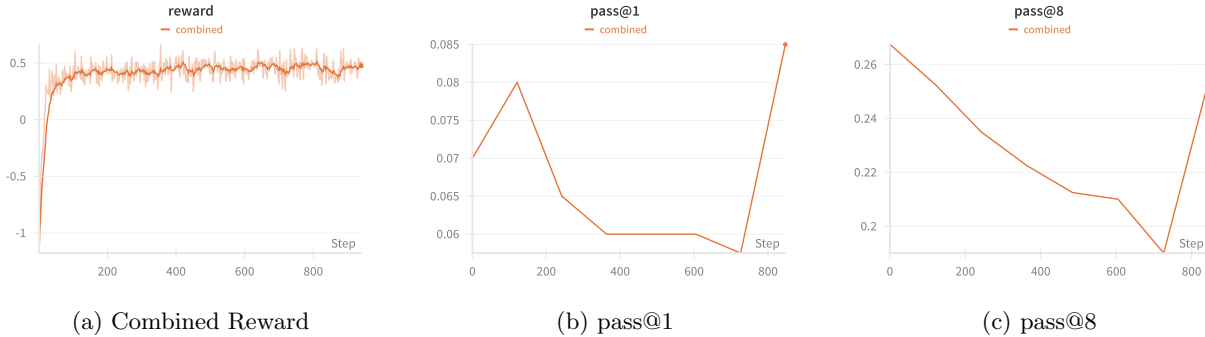


Figure 5: Training curves for the `combined` reward function showing reward, pass@1, and pass@8.

The training curves in Figure 5 reveal a fundamentally different optimization trajectory compared to the baseline RL run.

- **Reward Stabilization:** Unlike the high variance and severe mid-training collapse observed in the baseline RL run, the `combined` reward curve is exceptionally stable. Initialized with negative values (due to the strict formatting and hallucination penalties), the model rapidly adapts, climbing to a stable plateau around +0.5 by step 100. The continuous distance penalty and strict structural constraints successfully prevented the model from exploiting the reward signal, resulting in a tightly bounded, stable training run.
- **Evaluation Curve Volatility:** Curiously, while the internal reward remained stable, the evaluation metrics (pass@1 and pass@8) exhibited a steady decline through the mid-training phase before sharply recovering at the very end of the run (around step 800). This suggests a prolonged phase of competing gradients, where the model struggled to simultaneously satisfy the strict formatting/grounding penalties and output the correct mathematical integer.

Evaluation Metrics: The Reversal of the Alignment Tax

Table 7 presents the final benchmark results across the evaluation suite.

Task	SFT Initialization	Baseline RL (binary)	Combined RL
ARC-Easy	0.5497	0.5189	0.5501
ARC-Challenge	0.4394	0.4130	0.4343
MMLU	0.3675	0.3511	0.3628
GSM8K	0.1130	0.2252	0.0879
HumanEval	0.0122	0.0000	0.0793
SpellingBee	0.9961	0.0000	0.8398
ChatCORE	0.3217	0.1560	0.3006

Table 7: Comparison of the `combined` reward function model against the baseline RL run and the original SFT initialization.

The evaluation metrics demonstrate that our complex reward system successfully mitigated the catastrophic forgetting observed in Part Three, but introduced a new trade-off:

- **Recovery of General Capabilities:** The `combined` reward system successfully protected the model’s general reasoning and conversational capabilities. The heavy penalties for repetition, length, and formatting degeneration prevented the policy collapse that previously destroyed the model’s performance

on non-math tasks. Consequently, performance on HumanEval jumped to 0.0793 (surpassing even the SFT baseline), SpellingBee recovered to 0.8398, and the overall ChatCORE metric rebounded to 0.3006.

- **Over-Constrained Mathematical Reasoning:** The cost of this structural stability was a complete failure to improve on the target task. The GSM8K score dropped to 0.0879, which is significantly lower than the baseline RL run (0.2252) and even lower than the SFT starting point (0.1130).

Discussion

The **combined** reward function effectively acted as a strong regularizer. By severely penalizing hallucinated numbers and deviations from expected formatting, the reward function forced the model into a highly conservative generation mode. While this successfully maintained the "alignment" of the model (preserving conversational fluency and coding ability), it heavily penalized the exploratory "chain-of-thought" generation required to solve complex GSM8K problems. The model ultimately learned that generating safe, well-formatted, but mathematically incorrect text yielded a higher net reward than risking heavy hallucination penalties to arrive at the correct answer.

Future iterations of this reward system should likely reduce the negative weight of the **hallucination** and **parsimony** penalties, allowing the model more freedom to explore mathematical scratchpad reasoning without immediately triggering a negative reward cascade.

4.3 Analysis for each individual reward function

To isolate the effects of our proposed reward mechanisms, we first evaluate them independently before analyzing the combined approach. We begin with the **distance** reward system.

Distance-Based Accuracy Reward

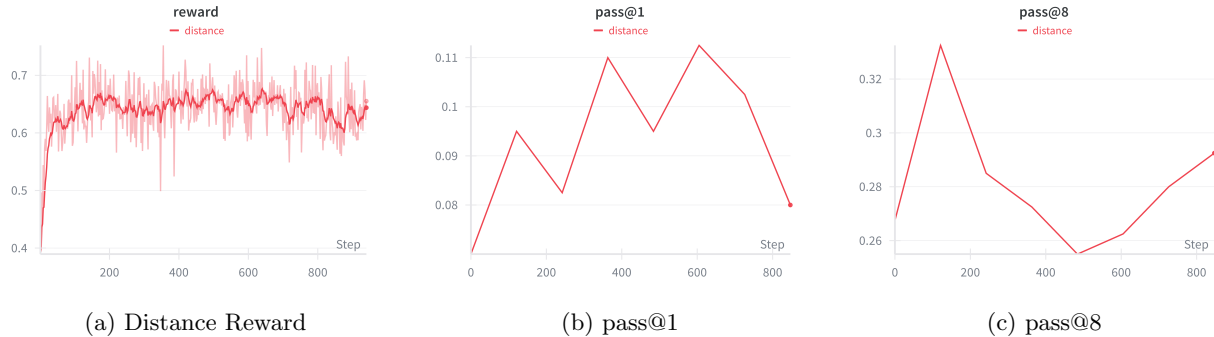


Figure 6: Training curves for the **distance** reward function showing reward, pass@1, and pass@8.

The **distance** reward replaces the sparse binary 0.0/1.0 signal with a continuous function based on relative numerical error, aiming to provide a smoother gradient for near-miss calculations.

Training Dynamics: Figure 6 shows a rapid initial increase in the reward curve, stabilizing around 0.65 without the catastrophic mid-training collapse observed in the binary baseline. However, the evaluation metrics (pass@1 and pass@8) tell a different story. While the model quickly learned to maximize the distance reward, the actual accuracy on GSM8K fluctuated significantly, with pass@1 peaking at roughly 0.11 before declining, and pass@8 showing a sharp drop mid-training before a slight recovery.

Evaluation Metrics: Table 8 presents the final benchmark results for the distance-based model.

Task	SFT Initialization	Baseline RL (binary)	Distance RL
ARC-Easy	0.5497	0.5189	0.5303
ARC-Challenge	0.4394	0.4130	0.4215
MMLU	0.3675	0.3511	0.3657
GSM8K	0.1130	0.2252	0.1092
HumanEval	0.0122	0.0000	0.0244
SpellingBee	0.9961	0.0000	0.1875
ChatCORE	0.3217	0.1560	0.1796

Table 8: Comparison of the **distance** reward function model against the baseline RL run and the original SFT initialization.

Discussion – The "Partial Credit" Trap

The results highlight an unintended consequence of continuous rewards in exact-match reasoning tasks. The distance reward failed to improve GSM8K performance, yielding a score of 0.1092 (slightly worse than its SFT initialization of 0.1130, and significantly worse than the binary RL’s 0.2252). By granting "partial credit" for numbers that are close in magnitude to the ground truth, the reward function inadvertently created a local optimum. The model learned that outputting a numerically similar answer—perhaps by skipping a final scaling step or using an easier, incorrect arithmetic operation—was a safer way to secure a ~ 0.8 reward than attempting a complex, multi-step calculation that risked generating a wildly wrong number and a near-zero reward.

Interestingly, because the model did not hyper-optimize for exact mathematical generation, it suffered less from catastrophic forgetting. It retained some ability in SpellingBee (0.1875) and actually improved on HumanEval (0.0244), suggesting that the softer reward landscape exerted less destructive pressure on the model’s general language distribution.

Reasoning and Anti-Degeneration Reward

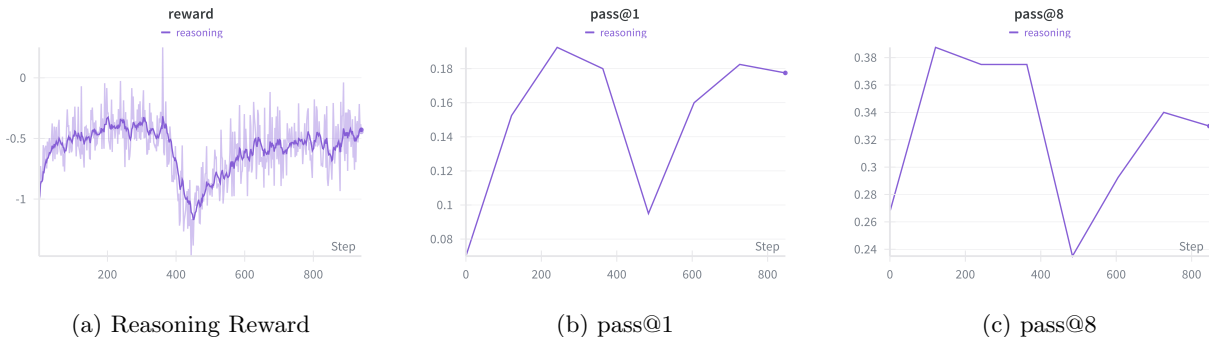


Figure 7: Training curves for the **reasoning** reward function showing reward, pass@1, and pass@8.

The **reasoning** reward system maintained the strict $+1.0/-1.0$ binary signal for mathematical accuracy but introduced structural penalties to combat formatting degeneration, repetitive n-gram loops, and excessive verbosity (parsimony penalty).

Training Dynamics: As seen in Figure 7, the reward operates entirely in the negative domain, reflecting the constant downward pressure of the length and formatting penalties. Strikingly, despite the explicit anti-degeneration constraints, this model exhibited a pronounced mid-training policy collapse that closely mirrors the baseline binary run. Between steps 350 and 500, the reward sharply drops below -1.0 , accompanied by severe crashes in both pass@1 and pass@8 evaluation metrics, before ultimately recovering. This suggests that formatting and repetition penalties alone are insufficient to prevent the policy instability inherent in

sparse, binary-driven RL optimization.

Evaluation Metrics: Table 9 details the final evaluation metrics for this model.

Task	SFT Initialization	Baseline RL (binary)	Reasoning RL
ARC-Easy	0.5497	0.5189	0.5168
ARC-Challenge	0.4394	0.4130	0.4224
MMLU	0.3675	0.3511	0.3556
GSM8K	0.1130	0.2252	0.1888
HumanEval	0.0122	0.0000	0.0122
SpellingBee	0.9961	0.0000	0.0039
ChatCORE	0.3217	0.1560	0.1552

Table 9: Comparison of the **reasoning** reward function model against the baseline RL run and the original SFT initialization.

Discussion – The Cost of Parsimony

The **reasoning** reward function achieved a GSM8K score of 0.1888. While this is a clear improvement over the SFT initialization (0.1130), it underperforms the pure binary baseline (0.2252). This degradation in peak mathematical performance is highly likely a direct consequence of the *parsimony penalty* (-0.001 per token).

In Large Language Models, successfully solving GSM8K problems heavily relies on generating extensive "Chain-of-Thought" (CoT) reasoning to act as a computational scratchpad. By penalizing the model for verbosity, the reward system inadvertently discouraged the very mechanism the model needs to process complex multi-step math problems.

Furthermore, the alignment tax remained severe. The structural penalties failed to protect the model’s general capabilities; performance on SpellingBee was almost entirely eradicated (0.0039), and the ChatCORE metric (0.1552) remained effectively identical to the collapsed baseline. This indicates that the strict ± 1.0 binary math signal still overwhelmingly dominated the policy gradient, forcing the model to sacrifice general language distribution in pursuit of mathematical accuracy.

Hallucination Reward

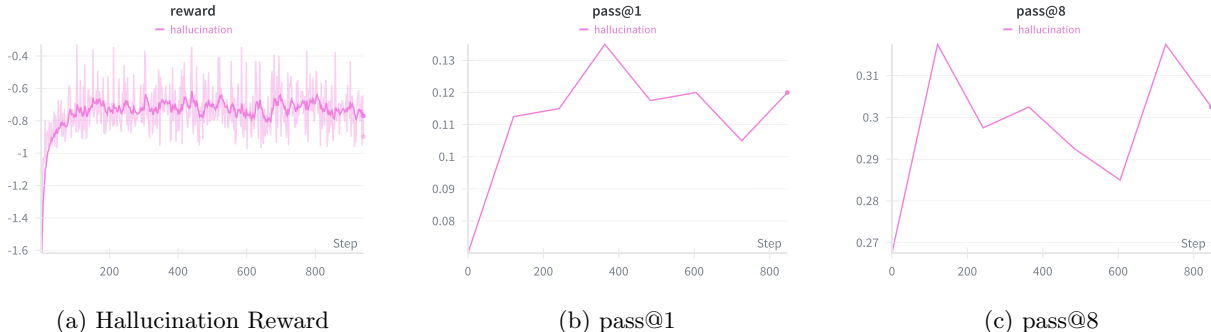


Figure 8: Training curves for the **hallucination** reward function showing reward, pass@1, and pass@8.

The **hallucination** reward system was specifically engineered to address the *missing constraint* and *unit-rate scaling* errors identified in Part 3.2. It acts as a strict computational grounder by seeding a pool of allowed numbers from the user’s prompt and penalizing the model (-0.5) for generating any untraceable operand within a calculation block.

Training Dynamics: As illustrated in Figure 8, the training dynamics for this reward function are remarkably stable. The reward curve quickly settles into a tight equilibrium bounded between -0.6 and -0.8 . Furthermore, the pass@1 and pass@8 curves avoid the catastrophic mid-training crashes that characterized the binary and reasoning runs. The strict grounding constraint effectively bounded the model’s exploratory generation space, preventing it from diverging into degenerate policy collapse.

Evaluation Metrics: Table 10 details the final evaluation metrics for this model.

Task	SFT Initialization	Baseline RL (binary)	Hallucination RL
ARC-Easy	0.5497	0.5189	0.5337
ARC-Challenge	0.4394	0.4130	0.4326
MMLU	0.3675	0.3511	0.3609
GSM8K	0.1130	0.2252	0.1175
HumanEval	0.0122	0.0000	0.0183
SpellingBee	0.9961	0.0000	0.3477
ChatCORE	0.3217	0.1560	0.2088

Table 10: Comparison of the `hallucination` reward function model against the baseline RL run and the original SFT initialization.

Discussion – The Missing Constants Problem

While the grounding reward successfully regularized the training process and mitigated catastrophic forgetting—retaining a SpellingBee score of 0.3477 and an improved HumanEval score of 0.0183—it entirely failed to improve the target GSM8K metric. The final GSM8K score of 0.1175 represents effectively zero progress over the SFT initialization (0.1130).

This stagnation exposes a critical flaw in strict operand tracking. Many grade-school math problems require the introduction of external, implicitly known mathematical constants that are not explicitly stated in the prompt (e.g., converting 1 hour to 60 minutes, using 100 to calculate a percentage, or knowing there are 7 days in a week). By applying a heavy -0.5 penalty to *any* number not present in the initial question or previous output block, the reward function actively punished the model for introducing these necessary constants. Consequently, the model was forced to abandon valid solution paths for any problem requiring external unit-rate conversions, artificially capping its performance.

4.4 How the Rewards Changed the Types of Mistakes

Comparison of Mistake Types Across RL Runs

Beyond comparing overall reward and evaluation performance, it is also useful to examine whether the additional rewards changed the types of mistakes made by the model. The two RL runs are therefore compared along three dimensions: overall error-type composition, error rates across problem types, and error rates across reasoning structures. Together, these visualizations reveal not only whether the model improved, but also which aspects of its error profile were most affected.

Figure 9 compares the proportions of major error types between the Original RL run and the RL run with additional rewards. Rather than uniformly reducing all mistakes, the additional rewards shifted the models error profile. In particular, wrong starting point, unit-rate scaling, relation translation, and generation format errors all became less common, suggesting better-structured and more controlled reasoning traces. However, arithmetic computation errors increased sharply, while missing constraint errors remained largely unchanged and continued to dominate. Overall, the additional rewards appear to improve surface-level reasoning organization without fully addressing the deeper challenges of constraint tracking and exact numerical execution.



Figure 9: Distribution of error types for incorrect GSM8K predictions.

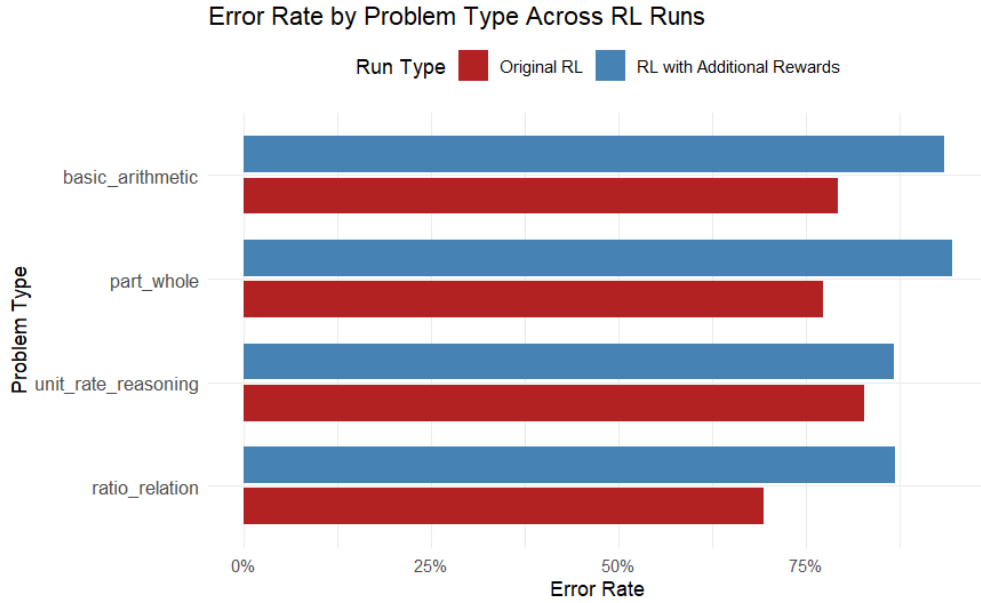


Figure 10: Error rates by problem type for the Original RL run and the RL run with additional rewards.

Figure 10 compares error rates across problem types between the Original RL run and the RL run with additional rewards. The pattern is consistent across all four categories: the additional-reward run has a higher error rate than the Original RL run. The gap is most noticeable for ratio/relation and part-whole problems, while unit-rate reasoning shows the smallest increase. Basic arithmetic also becomes somewhat more difficult under the new reward setting. This suggests that the additional rewards did not translate into better exact problem-solving performance at the problem-type level, even if they may have altered the model’s reasoning behavior in other ways.



Figure 11: Error rates by reasoning structure for the Original RL run and the RL run with additional rewards.

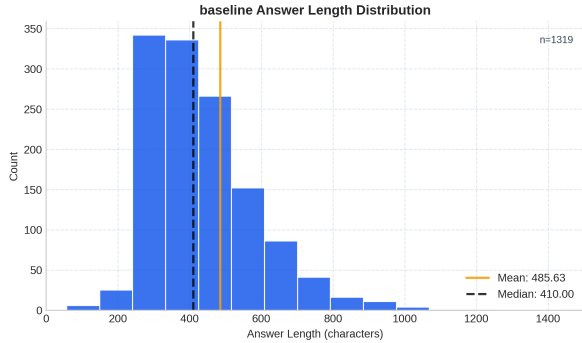
Figure 11 shows that the effect of the additional rewards differs across reasoning structures. For most categories, including sequential multi-step, change-tracking, group-aggregate, and single-step reasoning, the additional-reward run has a higher error rate than the Original RL run. The only exception is reverse reasoning, where the error rate decreases noticeably. This pattern suggests that the new reward design may provide some benefit for backward or constraint-oriented inference, but does not consistently improve the model’s performance on more standard forward reasoning structures.

Overall, these comparisons suggest that the additional rewards changed the model’s error profile more than its exact problem-solving ability. Structure-related failures, such as wrong starting points, relation translation mistakes, and formatting errors, became less common, indicating more organized reasoning traces. However, these gains did not reduce error rates across problem types and appeared only in a narrow subset of reasoning structures, especially reverse reasoning. Instead, the remaining failures became more concentrated in arithmetic computation and missing constraint errors. This is consistent with the earlier finding that the combined reward produces longer and numerically closer responses without improving exact-match accuracy: the reward shaping changes how the model fails more than how often it succeeds.

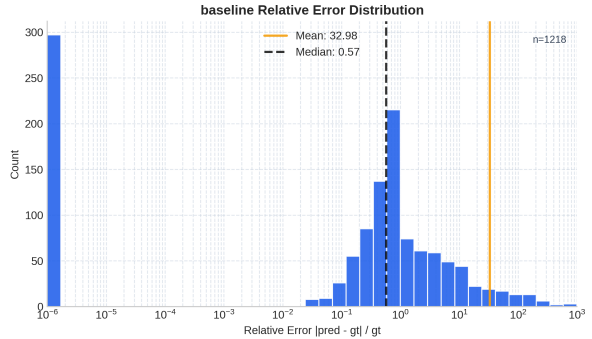
This pattern is consistent with the reward design itself: structural and grounding penalties directly suppress poorly formed or weakly grounded reasoning, but provide little support for long exact computation chains and may even over-constrain exploration.

One limitation of the analysis is that the error categories are necessarily coarse, and some responses may involve overlapping failure modes. The comparisons are therefore most useful for identifying broad shifts in error patterns rather than making fine-grained causal claims about every individual prediction.

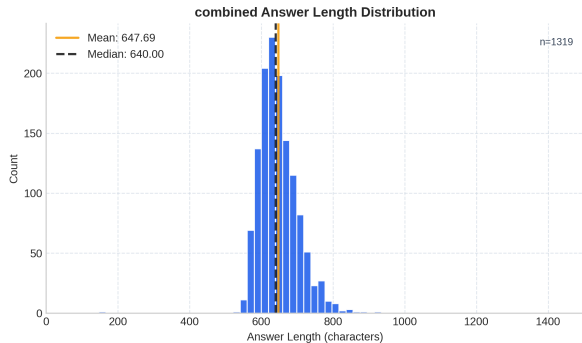
Behavioral Effects of the Combined Reward System



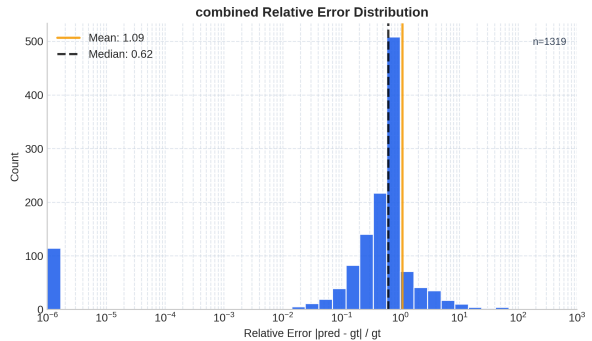
(a) Baseline answer length distribution



(b) Baseline relative error distribution



(c) Combined reward answer length distribution



(d) Combined reward relative error distribution

Figure 12: Distribution of answer length and relative numerical error for the baseline binary reward and the combined reward system. The combined reward produces significantly longer responses but smaller relative numerical errors, indicating that the model often generates lengthy or unstable reasoning traces even when the predicted answer is numerically closer to the ground truth.

To better understand how the reward design influences model behavior, we analyze the distributions of answer length and relative numerical error for the baseline binary reward and the combined reward system (Figure 12). The combined reward produces substantially longer outputs, with an average answer length of 647.69 characters compared to 485.63 for the baseline model. This suggests that the multi-objective reward encourages the model to generate longer reasoning traces and occasionally leads to unstable or repetitive outputs.

At the same time, the combined reward significantly reduces the relative numerical error (mean 1.09 compared to 32.98 for the baseline), indicating that predictions are often numerically closer to the correct answer even when they are not exactly correct. This behavior is consistent with the reward shaping design: the distance-based component encourages approximate correctness, while structural penalties influence reasoning format. However, because GSM8K evaluation requires an exact final answer, these improvements in numerical proximity do not translate into higher task accuracy.

4.5 Impact of Each Reward Design

Table 11 summarizes the GSM8K performance across all reward systems. Interestingly, the baseline binary reward achieves the highest accuracy (0.2252), outperforming all modified reward functions. This suggests that, despite its sparsity, the binary reward provides the clearest optimization signal because it aligns perfectly with the evaluation metric, which requires an exact numerical match.

The alternative reward functions introduce additional structure or intermediate objectives, such as distance-based numerical accuracy, reasoning format constraints, and grounding penalties. While these modifications were designed to improve training signal quality and reduce undesirable behaviors such as hallucination loops, they also introduce additional noise and competing objectives. As a result, the policy gradient may optimize for auxiliary properties (e.g., shorter reasoning traces or grounded calculations) rather than the exact final answer required by GSM8K evaluation.

The combined reward function exhibits the lowest performance (0.0879), illustrating the potential downside of overly complex reward shaping in small-scale RL training. When multiple objectives are combined, the effective reward landscape becomes noisier and the gradient signal less aligned with the primary task. Overall, these results highlight that for well-defined tasks such as GSM8K, a simple and highly aligned reward signal can outperform more elaborate reward shaping strategies.

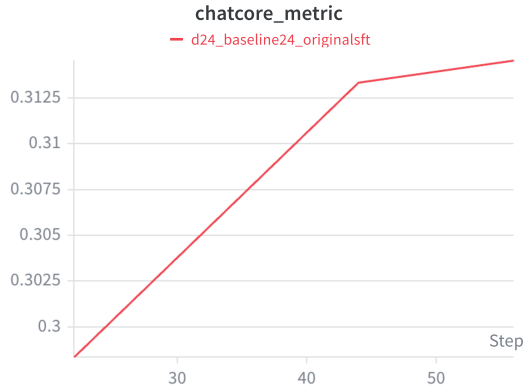
Reward System	GSM8K Score	Key Observations
Binary (Baseline)	0.2252	Uses a strict correctness signal (1 for exact match, 0 otherwise). Although the reward is sparse, it provides a clean and highly aligned objective with the GSM8K evaluation metric. In practice, this setting produced the most stable outputs and the highest numerical accuracy among all reward systems.
Distance-Based Accuracy	0.1092	Introduces a continuous reward based on relative numerical error to provide smoother feedback for near-correct answers. However, this weaker objective appears to dilute the pressure to reach the exact answer. In several cases the model produced repetitive reasoning segments and repeated answer markers (e.g., multiple “####” lines), suggesting that the model may exploit intermediate reward signals without converging to a stable final output.
Reasoning & Anti-Degeneration	0.1888	Adds structural incentives for producing well-formed answers and penalties for repetition loops and excessive verbosity. This helps reduce some degenerate behaviors such as runaway text generation, and generally improves response structure. However, these structural objectives do not directly optimize numerical correctness, so the final GSM8K accuracy remains slightly below the baseline.
Computational Grounding (Hallucination)	0.1175	Penalizes the use of numbers that cannot be traced back to the problem statement or previous calculations. While this encourages more grounded reasoning steps, it may also over-constrain the model and penalize otherwise recoverable reasoning paths, resulting in lower task accuracy.
Combined Reward	0.0879	Integrates all previous objectives into a multi-objective reward signal. Instead of improving behavior, this combination introduces significant reward noise and conflicting optimization targets. Empirically, responses sometimes exhibited degraded formatting, including extremely long repeated character sequences or failure to terminate cleanly after the answer, indicating unstable training dynamics.

Table 11: Comparison of GSM8K performance under different reward systems.

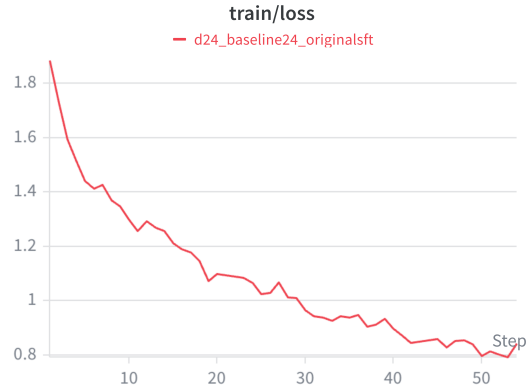
References

Shao, Zhihong, Peiyi Wang, Zhen Chen, et al. (2024). “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models”. In: *arXiv preprint arXiv:2402.03300*.

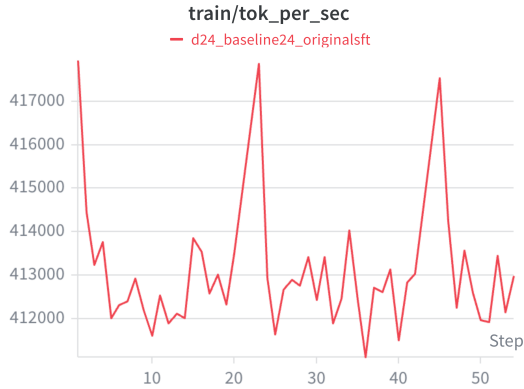
5 Appendix



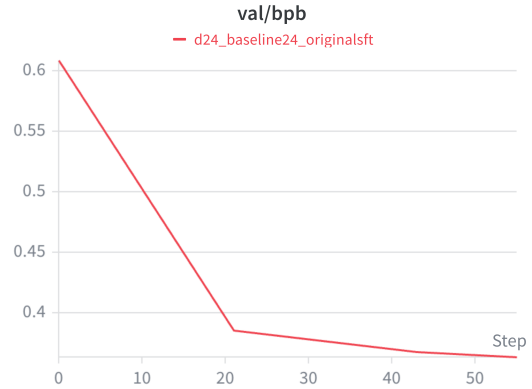
(a) Chatcore evaluation metric



(b) Training loss



(c) Tokens per second



(d) Bits per byte

Figure 13: Training logs for the depth-24 NanoChat baseline during supervised fine-tuning (SFT) with the original datasets. Metrics shown include training loss, ChatCORE evaluation, tokens per second (TPS), and bits per byte (bpb).